Add icon Add cover

Elastic certified engineer exam - 26.02.26

- 기출문제와 9문제가 유사하게 나왔습니다. 문제 뽑기 운이 조금 있을 듯합니다.
- 어떤 주제에 관한 문제인지 나옵니다 (1. Index lifecycle management). 혹시 생각이 안나신다면 해당 키워드로 공식문서에서 검색하시면 될 듯합니다.
- 공식문서 페이지는 링크와 같았습니다.
- 대부분은 search query 전체 다 적으라고 합니다, 다만 결과 중 일부만 적으라는 문제가 있을 수 있으니 조심하세요
- 맥 사용자 분들은 command가 아닌 control을 써야 합니다. (ctrl +c, v...) 전체 search query를 붙여넣으라는 문제가 많으니 유의하시길 바랍니다.

Task #1 (Index lifecycle management)

- cluster1에 해당 조건에 맞는 Data stream을 생성하시오.
  - mylogs-*-*패턴을 갖고 task1이라는 이름의 Index template 생성
  - Hot node에는 5분 동안 머물러 있고, rollover 시점엔 즉시 Warm node로 이동함
  - Warm node에 3분 동안 머물러 있고 Cold node로 이동함
  - Rollover 후 6분 뒤에 삭제함
  - 위 조건들에 해당되는 Index Template을 다 만들면 다음 쿼리를 실행하시오.
    - POST mylogs-system.env-prod/_doc { "@timestamp": "2022.08.15T01:00:59.123Z", "message": "This is sample message." }

(kibana ui에서 index lifecycle policy 만드시고 index template의 setting에서 "index.lifecycle.name"으로 적용)

Task #2

- cluster1의 food-ingredients 인덱스에 다음 쿼리를 입력하면 n개의 결과가 나옴

JSON

```
GET food-ingredients/_search
{
  "query": {
    "match": {
      "ingredients": "for"
    }
  }
}
```

- cluster1에 해당 조건에 맞는 인덱스를 생성하시오.
  - 인덱스의 이름은 task2로 지정
  - ingredients 필드는 text only, weight 필드는 keyword only
  - 다른 mapping들은 그대로
  - task2 인덱스로 food-ingredients의 document를 reindex함
  - reindex 후, 위 검색 쿼리를 task2에 적용했을 때, hits 값이 0이어야 함

JSON

```
#제출답안

PUT task2
{
  "settings": {
    "analysis": {
      "analyzer": {
        "task2_analyzer": {
```

```
          "tokenizer": "standard",
          "filter": ["task2_filter"]
        }
      },
      "filter": {
        "task2_filter": {
          "type": "stop",
          "ignore_case": true,
          "stopwords": ["for"]
        }
      }
    }
  },
  "mappings": {
    "properties": {
      "ingredients": {
        "type": "text",
        "analyzer": "task2_analyzer"
      },
      "weight": {
        "type": "keyword"
      }
    }
  }
}

POST _reindex
{
  "source": {
    "index": "food-ingredients"
  },
  "dest": {
    "index": "task2"
  }
}

GET task2/_search
{
  "query": {
    "match": {
      "ingredients": "for"
    }
  }
}


```

Task #3 (Ingest pipeline)

- cluster1의 inspections 인덱스엔 주소 정보를 가진 복수의 필드가 있음
  - street_number, street_name, neighborhood, zipcode
- 해당 조건에 맞게 inspections 인덱스를 업데이트하시오.
  - street_number, street_name, neighborhood, zipcode 값이 순서대로 연결된 문자열 형태로 complete_address 필드의 값에 저장이 되어야 함
  - 예
    - street_number: 168
    - street_name: 죽전로
    - neighborhood: 아무개
    - zipcode: 16876
    - complete_address: 168 죽전로 아무개 16876

JSON

```
#제출답안
#저는 script update_by_query로 풀었습니다. 주제가 ingest pipeline이라고 적혀있었어서 감점이 있을지도 모르

```

```
POST inspections/_update_by_query
{
  "script": {
    "source": "ctx._source.complete_address = ctx._source.street_number + ' ' + ctx._sour
    "lang": "painless"
  }
}

# ingest pipeline으로 푼다면 아래와 같이 풀면 될듯 합니다

PUT _ingest/pipeline/address_join_pipeline
{
  "processors": [
    {
      "set": {
        "field": "complete_address",
        "value": "{{street_number}} {{street_name}} {{neighborhood}} {{zipcode}}"
      }
    }
  ]
}

POST inspections/_update_by_query?pipeline=address_join_pipeline
```

Task #4 (Asynchronous search)

- cluster1의 flights 인덱스를 활용하여 다음 조건에 맞는 Async Search 쿼리를 입력하시오.
  - cluster1의 flights 인덱스에서 주별(Weekly) 평균 DistanceInMiles를 구해야 함
  - hits는 보이지 않아야 함

JSON

```
#제출답안
#저는 async search를 급하게 공식문서에서 찾아봤었습니다. lab 3.3에 있습니다
#/_mapping으로 type이 Date인 field를 찾아보셔야 합니다.

POST flights/_async_search
{
  "size": 0,
  "aggs": {
    "mile_week": {
      "date_histogram": {
        "field": "FlightDate",
        "calendar_interval": "week"
      },
      "aggs": {
        "average": {
          "avg": {
            "field": "DistanceInMiles"
          }
        }
      }
    }
  }
}

```

Task #5 (Runtime fields)

- cluster1의 movie_data 인덱스를 활용하여 다음 조건에 맞는 쿼리를 입력하시오.
  - 검색 시, revenue 필드 값에서 budget 필드 값을 뺀 값이 담겨있는 profit 필드를 생성함
  - profit 필드를 이용해 다음 조건에 맞는 document 수를 구해야 함
    - profit < 0
    - 0 <= profit < 1000000
    - 1000000 <= profit
  - hits는 보이지 않아야 함

JSON

```
#제출답안

GET movie_data/_search
{
  "size": 0,
  "runtime_mappings": {
    "profit": {
      "type": "long",
      "script": """
        emit(doc['revenue'].value - doc['budget'].value)
      """
    }
  },
  "aggs": {
    "profit_range": {
      "range": {
        "field": "profit",
        "ranges": [
          {
            "to": 0
          },
          {
            "from": 0,
            "to": 10000000
          },
          {
            "from": 10000000
          }
        ]
      }
    }
  }
}
```

Task #6

- cluster1의 flight 인덱스에서 ActualElapsedTime의 평균값이 가장 높은 두 개의 Airline이 무엇인지 입력하시오.

JSON

```
#제출답안
#답안은 쿼리 전체가 아닌 AS, UA 처럼만 입력하도록 되어있었습니다.

GET flights/_search
{
  "size": 0,
  "aggs": {
    "high_avg": {
      "terms": {
        "field": "Airline",
        "order": {
          "average": "desc"
        }
      },
      "aggs": {
        "average": {
          "avg": {
            "field": "ActualElapsedTime"
          }
        }
      }
    }
  }
}
```

Task #7 (multi match)

- cluster1의 inspections 인덱스의 다음 조건에 충족하는 쿼리를 작성하시오.
  - category, item, item_description 필드에서 fire라는 값을 질의
  - score는 검색 조건에 해당된 필드들의 수

Task #8 (CCS)

- cluster1의 listings, cluster2의 listings2 인덱스에서 해당 조건에 맞는 쿼리를 작성하시오.
  - review_scores_rating 필드의 값이 95 이상
  - summary 필드에 "city center" phrase 질의

JSON

```
#제출답안
#cluster2 연결은 되어있습니다

GET listings,cluster2:listings2/_search
{
  "query": {
    "bool": {
      "must": [
        {
          "match_phrase": {
            "summary": "city center"
          }
        }
      ],
      "filter": [
        {
          "range": {
            "review_scores_rating": {
              "gte": 95
            }
          }
        }
      ]
    }
  }
}
```

Task #9

- cluster2에 해당 조건에 맞는 snapshot을 생성하시오.
  - Snapshot은 /mnt/elastic-snapshots 경로에 저장되어야 함
  - Repository 이름은 task9로 지정
  - 생성된 snapshot 이름은 city_population_1이어야 하며, 저장한 인덱스는 city_population만 있어야 함

(cluster1 / cluster2 구분 조심. snapshot repository가 path.repo 문제로 안 만들어진다면 cluster2인지 확인해보세요...)

Task #10 (sort, pagination)

- cluster1의 **movie_data** 인덱스를 활용하여 다음 조건에 맞는 검색 쿼리를 작성하시오.
  - **title** 필드에 **star 질의**.
  - 검색 결과 중 **상위 15개의 문서는 제외하고, 그 뒤를 잇는** 5개의 문서만 출력되어야 함.
  - 결과는 **revenue** 필드 값을 기준으로 **descending** 정렬하고, 만약 값이 같다면 **_score**를 기준으로 다시 정렬해야 함.

JSON

```
#제출답안
#_score는 asc 정렬인지, desc정렬인지 문제에서 보이진 않았습니다. 아마 desc이지 않을까 싶습니다

GET movie_data/_search
```

```
{
  "from": 15,
  "size": 5,
  "sort": [
    {
      "revenue": {
        "order": "desc"
      }
    },
    {
      "_score": {
        "order": "desc"
      }
    }
  ],
  "query": {
    "match": {
      "title": "star"
    }
  }
}
```